Uganda Christian University
Faculty of Engineering, Design and Technology
Department of Computing and Technology
Opol Charis Reg. No. S24B23/003 B29654
Concept Note: Implementation Proposal

1. Proposed Project Title

Investigate, Recommend or Escalate: A Rule-Constrained Reinforcement Learning Agent for IT Incident Triage under Ambiguous Evidence

2. Introduction and Problem

When an IT incident is reported, staff must decide what to do before they know its cause. The first evidence (an error in the logs, a user's ticket, a dashboard alert) is often incomplete, unreliable or consistent with several causes. Staff then face three choices: investigate further, apply a likely fix, or hand the incident to someone better placed. Each has a cost. Investigation takes time, a wrong fix can prolong or worsen an outage, and unnecessary escalation consumes senior staff. At the Directorate of University ICT Services (UCU), tickets are vetted and assigned to the responsible developer, or to a senior developer when the owner is unknown, and application errors are diagnosed from API logs and a monitoring dashboard. Existing AI tools for IT operations (AIOps) treat triage as a single prediction from the ticket: they cannot ask for more evidence, cannot say that they are unsure, and do not check their suggestions against the organisation's operating rules.

3. Research Gap

Three bodies of work each address part of the problem. *Information acquisition* lets an agent choose between looking further and deciding; *learning to defer* lets a model choose between deciding and handing over to a human; *shielding* stops a learning agent from executing actions that break set rules, but has been shown mainly in robotics and games. In the 46 sources reviewed, these ideas appear at most two at a time. No study puts all three choices (investigate, recommend, escalate) in one agent for IT incidents, enforces operating rules on it, and measures how its decisions change as the evidence gets worse.

4. Proposed Solution

A reinforcement learning (RL) agent that, for each incident, learns when to **request a diagnostic check**, when to **recommend a fix** the rules permit, and when to **escalate** to a human expert. An independent rule-based safety mechanism checks every proposal before it takes effect. The agent is trained and evaluated in a simulator built from incident scenarios elicited from the Directorate, in which the level of ambiguity can be controlled. It is never connected to university systems.

5. Proposed Methodology and Implementation

The study follows a design-and-evaluate approach in eight phases: elicit scenarios → build and validate the environment → encode the rules → implement baselines → train agents → run experiments → analyse.

5.1 Scenario model (input data)

To be elicited from Directorate staff in the week of 26 October 2026, using a structured workbook. Target scale: **about 15 causes** in four categories (application and interface; authentication and access; database; infrastructure and external services), each with one correct fix; **20 diagnostic checks** (e.g. an API log entry, a dashboard metric, reproducing a failing request, a service's status), each with an effort cost; a fixed list of symptom indicators; and **10 operating rules**. For each cause–check pair, staff will rate how often the check is positive on a five-point scale (never, rarely, sometimes, often, almost always ↦ 0.05, 0.25, 0.5, 0.75, 0.95). No logs, tickets or personal data enter the simulator. Directorate staff will then review sample incidents, and the model will be frozen *before* any training.

5.2 Environment

A Gymnasium environment modelling triage as a partially observable decision problem; one episode is one incident.

- **Hidden state:** a cause $c \sim P(c)$ and a visible context (production system or not, priority). Check results are drawn once from $\theta_{ij} = P(d_i{=}\text{positive} \mid c_j)$, so repeating a check returns the same result.
- **Observation:** one-hot vector of each check's status (not requested / unavailable / negative / positive), symptoms (present / absent / not reported), context, normalised step count, and a *blocked* flag.
- **Actions (36):** 20 *request check* d_i, 15 *recommend fix for* c_j, 1 *escalate*. A recommendation or escalation ends the episode; reaching the step limit counts as an escalation.
- **Reward:** $+r_{\text{correct}}$ for a correct fix, $-\lambda_{\text{wrong}}$ for a wrong fix, $-k_i$ per check, $-\lambda_{\text{esc}}$ per escalation, with $r_{\text{correct}} > -\lambda_{\text{esc}} > -\lambda_{\text{wrong}}$ (starting values +10, −3, −10; checks 0.1–1.0; tuned in pilot runs).
- **Ambiguity controls:** missing evidence $p_{\text{miss}} \in \{0, .2, .4, .6\}$ (check unavailable); unreliable evidence $p_{\text{noise}} \in \{0, .1, .2, .3\}$ (result flipped); overlapping evidence $\omega \in \{0, .25, .5, .75\}$, which blends each cause's profile towards its category mean, $\theta'_{ij} = (1-\omega)\theta_{ij} + \omega\bar{\theta}_i$.
- **Reference labels** (evaluation only, hidden from the agent): an incident "should be escalated" if the rules forbid the true fix in its context or if it stays ambiguous even with every check observed.

5.3 Rules and the safety mechanism

Each of the ten rules is a readable condition over *(proposed action, context, evidence so far)*, kept in a configuration file and separate from the agent. The rules are not yet known; illustrative forms only: (i) a fix is forbidden on a production system (e.g. restarting a production database); (ii) a fix is forbidden until a named check has been observed (e.g. no restore from backup before backup integrity is confirmed); (iii) actions that touch user accounts or access rights always need a human, reflecting the Directorate's data-security concern. Escalation is never forbidden, so a permitted action always exists. Procedure: the agent proposes → if permitted, it executes; if blocked, the event is logged, the agent sees the *blocked* flag with that action masked and chooses *once* more; if the second proposal is also blocked, the incident is **force-escalated** and reported separately.

5.4 Algorithm and training

Primary learner: **Deep Q-Network (DQN)** (Stable-Baselines3), chosen because the action set is small and discrete, the 4^{20} check combinations rule out tabular Q-learning, and per-action Q-values can be inspected (e.g. the value of a blocked fix vs. escalation). Invalid actions (repeated checks, blocked proposals) are masked both in action selection and in the max of the learning target. Secondary learner: **PPO** with action masking (sb3-contrib MaskablePPO) to test whether findings depend on the learner. DQN needs a function estimating $Q(s, a)$ for all 36 actions; a table is infeasible and a linear model cannot capture how evidence combines, so a small MLP is used (two hidden layers of 128 units). No recurrent model is needed, as the observation already summarises all evidence. Hyperparameters (learning rate, network size, exploration schedule) are tuned on seeds held out from the experiments; training length is set where mean return plateaus in pilots. Two regimes: **A**, training at a fixed base ambiguity level; **B**, ambiguity controls randomised per episode. Each agent is trained with **10 random seeds**. Tools: Python, Gymnasium, Stable-Baselines3, PyTorch.

5.5 Benchmarks (baselines)

All run in the same environment on the same evaluation incidents.

Policy	Behaviour (approach it represents)
Initial-symptom	Recommends the most likely cause from first symptoms, no checks (AIOps single prediction)
Scripted checklist	Fixed check sequence per category, then recommends (runbook practice)
Always-escalate	Hands every incident to a human (limit of deferral)
Confidence threshold	Bayesian posterior; most informative check until a cause passes a threshold, else escalates (strong reference: knows the model)
Unconstrained agent	Same DQN, no safety mechanism (standard shielding comparison)
Penalty-only agent	Same DQN, no mechanism, reward penalty per rule violation

Two ablations isolate each option: the agent *without checks* (recommend or escalate only) and *without escalation* (investigate then recommend).

5.6 Evaluation

Metrics. *Decision quality:* resolution accuracy, coverage, selective accuracy, wrong-fix rate, mean return. *Investigation efficiency:* mean checks, mean check cost, accuracy–cost curve. *Escalation behaviour:* escalation rate, precision, recall and F1 against the reference labels. *Constraint compliance:* proposed-violation rate, forced-escalation rate, executed-violation rate, compliant success rate. **Experiments.** E1: agent vs. six baselines at base ambiguity. E2: effect of the safety mechanism (vs. unconstrained and penalty-only). E3: action-set ablations. E4: robustness across all ambiguity levels, both training regimes. A sensitivity analysis varies λ_{esc}. Results are means over 10 seeds with 95% bootstrap confidence intervals, on evaluation seeds never used in training.

5.7 Expected results

H1: the agent achieves higher resolution accuracy than the initial-symptom, scripted and always-escalate policies at equal or lower investigation cost, and approaches the threshold policy without being given the model. **H2:** with the mechanism, executed violations are zero and compliant success is higher than for the unconstrained and penalty-only agents; the proposed-violation rate falls over training, showing the agent learns the rules rather than relying on being stopped. **H3:** as ambiguity rises the agent runs more checks and escalates more, rather than recommending on weaker evidence; regime B degrades less than regime A. Numerical targets are fixed after pilot runs, and the results are reported whether or not the hypotheses hold.

6. Research Contribution

(1) A controllable simulator of IT incident triage in which missing, unreliable and overlapping evidence can each be varied, grounded in a real university IT directorate. (2) The first agent, to our knowledge, that combines investigating, recommending and escalating in one policy under an independent, human-readable rule checker with a review-then-escalate fallback. (3) An evaluation that reports decision quality, investigation cost, escalation behaviour and rule compliance together across ambiguity levels, rather than accuracy alone.

7. Expected Prototype

A working simulator (about 15 causes, 20 checks, 10 rules), the rule-based safety mechanism, six baselines and trained DQN/PPO agents, with scripts that reproduce every experiment. A lightweight dashboard replays an incident step by step: the evidence revealed, the agent's Q-values, any blocked proposal, and the final decision, beside the comparison results. Feasibility: training runs on a laptop CPU; the Directorate has agreed to support elicitation and scenario review.